سوالات تحلیلی

سوال ۱: کاربرد بیت APRE در شمارنده چیست؟

TIMx control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

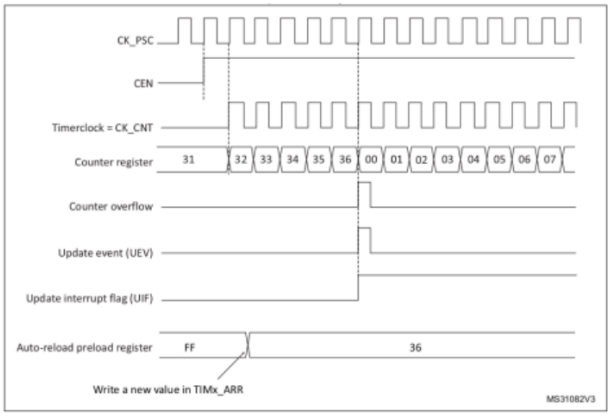
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

بیت APRE، بیت 8ام TIMx control register است.

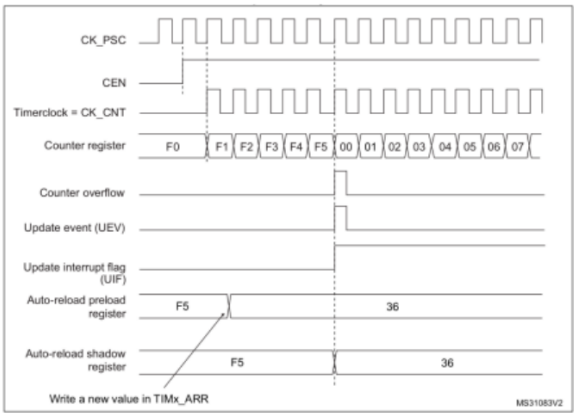
اگر این بیت clear باشد، مقدار preload register همواره در shadow register نوشته میشود.

اگر این بیت set باشد، مقدار preload register بعد از رسیدن به update event در shadow register نوشته میشود.

APRE=0



APRE=1

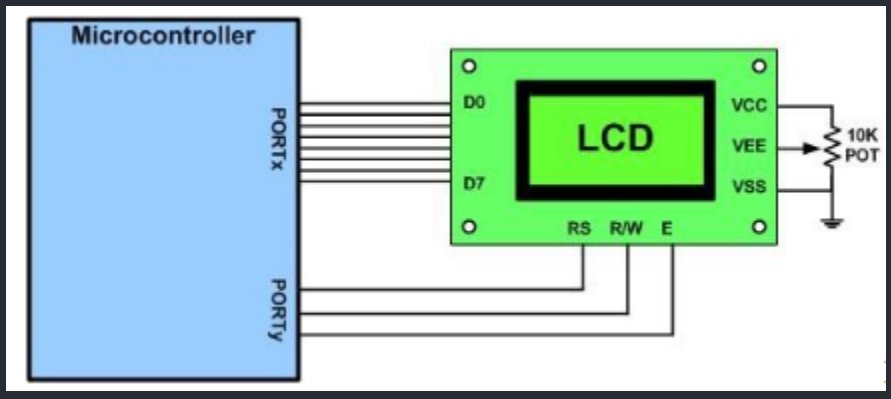


سوال ۲: روش کار شمارنده هنگامی که بیت DIR برابر 1 یا صفر است چیست؟

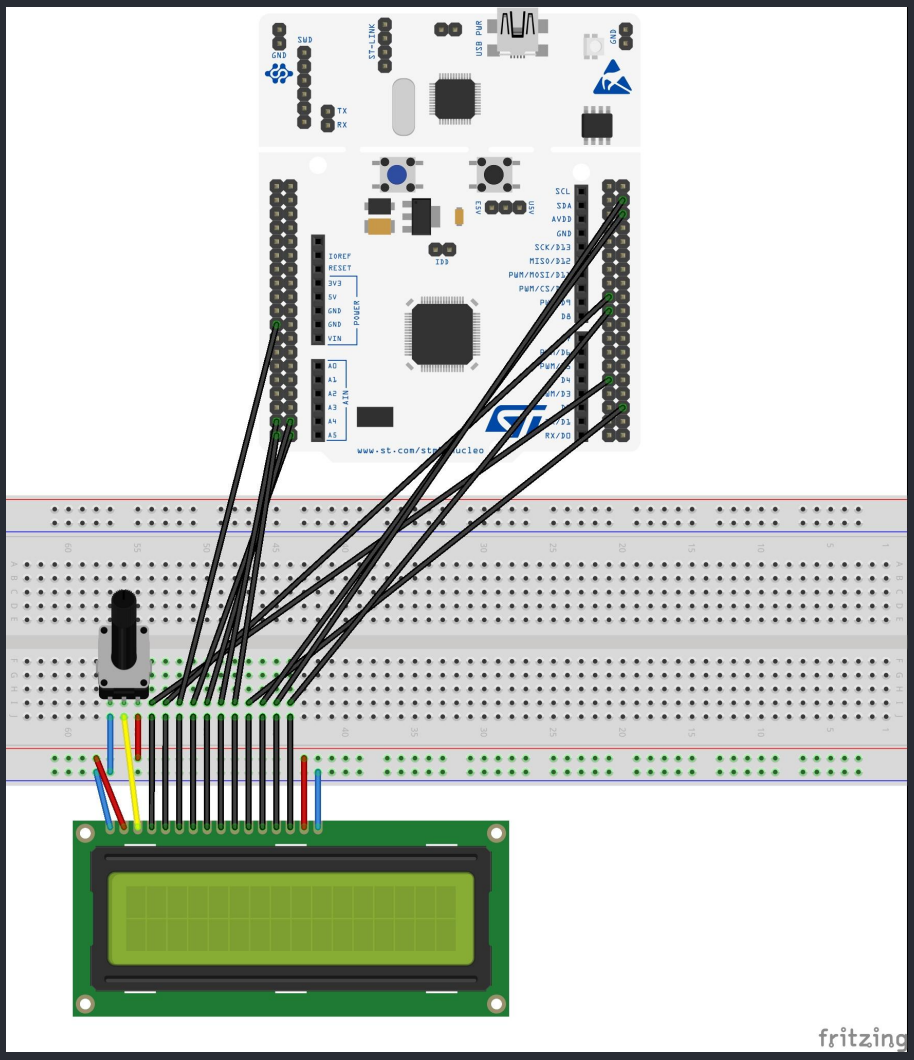
اگر این بیت clear باشد، شمارنده به سمت بالا میشمارد(count up mode). یعنی update event زمانی رخ میدهد که TIMx_CNT از 0 به مقدار TIMx_ARR برسد و دوباره update شود.

اگر این بیت set باشد، شمارنده به سمت پایین میشمارد (count down mode). یعنی update event زمانی رخ میدهد که TIMx_CNT از مقدار TIMx_ARR به 0 برسد و دوباره update شود.

سوال ۳: مقاومت متغیر مورد نیاز برای راه اندازی LCD ذکر شده چگونه متصل میشود و چه کاربردی دارد؟

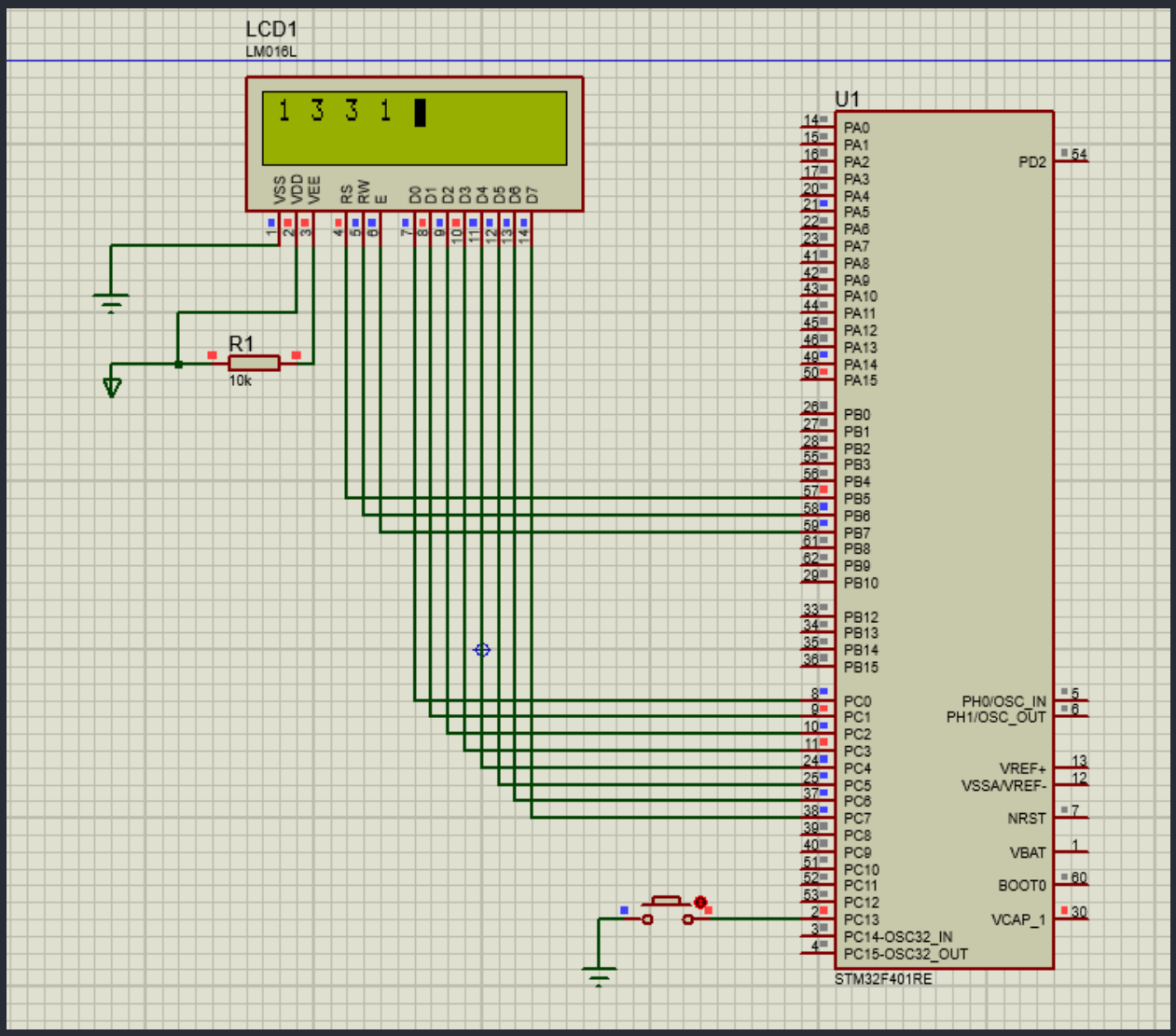


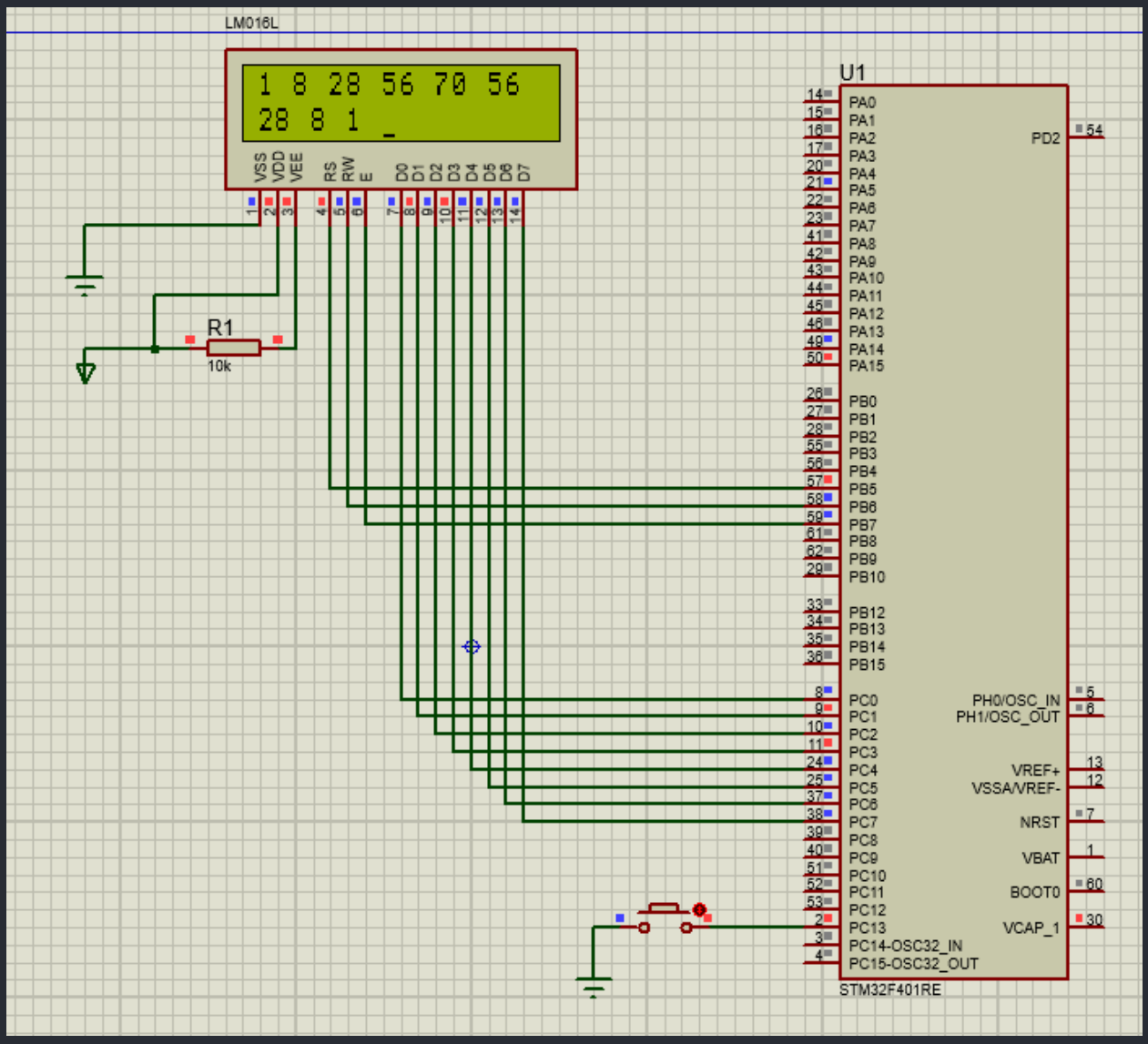
پین VEE یا V0 که معمولا به یک پتانسیومتر متصل میشود و این قابلیت را به کاربر میدهد که contrast صفحه نمایش را تنظیم کند.



گزارش دستور کار:

شمای مدار:




LM016L
1 8 28 56 70 56
28 8 1 _
VSS VDD VEE RS RW E D0 D1 D2 D3 D4 D5 D6 D7
R1
10k
U1
PA0 PA1 PA2 PA3 PA4 PA5 PA6 PA7 PA8 PA9 PA10 PA11 PA12 PA13 PA14 PA15
PB0 PB1 PB2 PB3 PB4 PB5 PB6 PB7 PB8 PB9 PB10 PB12 PB13 PB14 PB15
PC0 PC1 PC2 PC3 PC4 PC5 PC6 PC7 PC8 PC9 PC10 PC11 PC12 PC13 PC14-OSC32_IN PC15-OSC32_OUT
PD2
PH0/OSC_IN PH1/OSC_OUT
VREF+ VSSA/VREF-
NRST
VBAT
BOOT0
VCAP_1
STM32F401RE

کد برنامه:

برنامه شامل چندین فایل است،

main.c

main.c | LCD.c | LCD.h | GPIO.c | EXTI.c | EXTI.h

```
16  #include "stm32f4xx.h"
17  #include "LCD.h"
18  #include "GPIO.h"
19  #include "EXTI.h"
20
21  int pascal(int row, int col);
22  void LCD_print_KHPA(int n);
23  void TIM2_IRQHandler(void);
24  void EXTI15_10_IRQHandler(void);
25
26  float freq = 0.5; /* FREQUENCY OF LCD */
27  int line=1; /* LINE OF TRIANGLE*/
28
29  int main(void) {
30    LCD_init();
31
32    // Init SevenSegment
33    GPIO_EnableClock(B);
34    GPIO_EnableClock(C);
35
36    char i;
37    for (i = 5; i < 7; i++) {
38      GPIO_Init(B, i, OUTPUT, NO_PULL_UP_DOWN);
39    }
40    for (i = 0; i < 8; i++) {
41      GPIO_Init(C, i, OUTPUT, NO_PULL_UP_DOWN);
42    }
43
44    // Init UserButton
45    GPIO_Init(C, 13, INPUT, PULL_UP);
46
47    // Port C EXTI
48    EXTI_EnableClock();
49    EXIT_INIT(C, EXTI13, FALLING_RISSING);
50
51    //Enable Interrupt
52    __enable_irq();
53    NVIC_ConfigIRQ(EXTI15_10_IRQn, 0);
54
55    NVIC_EnableIRQ(TIM2_IRQn); /* enable interrupt in NVIC */
```

```
56    while(1) {
57      RCC->AHB1ENR |= 1; /* enable GPIOA clock */
58      GPIOA->MODER &= ~0x00000C00;
59      GPIOA->MODER |= 0x00000400;
60      /* setup TIM2 */
61      RCC->APB1ENR |= 1; /* enable TIM2 clock */
62      TIM2->PSC = 16000 - 1; /* divided by 16000 */
63      TIM2->ARR = 1000 / freq - 1; /* divided by 1000 */
64      TIM2->CR1 = 1; /* enable counter */
65      TIM2->DIER |= 1; /* enable UIE */
66    }
67  }
68
69  void TIM2_IRQHandler(void) {
70    TIM2->SR = 0; /* clear UIF */
71    LCD_clear();
72    LCD_print_KHPA(line);
73    line=(line+1)%9 == 0 ? 9:((line+1)%9);
74  }
75
76  void LCD_print_KHPA(int n) {
77    for (int i = 1; i <= n; i++) {
78      LCD_print_number(pascal(n, i));
79      LCD_print_string("\n");
80    }
81  }
82
83
84  int pascal(int row, int col) {
85    if (col == 1 || row == col)
86      return 1;
87    else
88      return pascal(row - 1, col) + pascal(row - 1, col - 1);
89  }
90
91
92  void EXTI15_10_IRQHandler(void) {
93    RCC->APB1ENR |= 0; /* disable TIM2 clock */
94    EXTI->PR |= EXTI_PR_PR13;
```

```
91
92  void EXTI15_10_IRQHandler(void) {
93    RCC->APB1ENR |= 0; /* disable TIM2 clock */
94    EXTI->PR |= EXTI_PR_PR13;
95    NVIC_ClearPendingIRQ(EXTI15_10_IRQn);
96    if(GPIO_ReadPin(C, 13) == 0) {
97      freq = (freq == 1) ? 0.5 : 1;
98    }
99    TIM2->SR = 0; /* clear UIF */
100   RCC->APB1ENR |= 1; /* enable TIM2 clock */
101
102 }
```

LCD.c

```c
#include "LCD.h"
#include "stm32f401xe.h"


int printed_length = 0;


void LCD_print_number(int num) {
  int digits = digit_count(num);
  char* str = (char*)malloc(digits * sizeof(char));
  sprintf(str, "%d", num);
  LCD_print_string(str);
}

void LCD_print_string(char str[]) {
  for(int i=0;; i++) {
    if(str[i] == '\0') break;
    LCD_data(str[i]);
  }
}

int digit_count(int num) {
  int count = 0;
  while (num != 0) {
      count++;
      num /= 10;
  }
  return count;
}


void LCD_clear(void) {
  LCD_command(1);
  printed_length=0;
}

/* Initialize port pins then initialize LCD controller */
void LCD_init(void) {
  PORTS_init();
  delayMs(30); /* initialization sequence */
```

```
40    delayMs(30); /* initialization sequence */
41    LCD_command_noPoll(0x30); /* LCD does not respond to status poll yet*/
42    delayMs(10);
43    LCD_command_noPoll(0x30);
44    delayMs(1);
45    LCD_command_noPoll(0x30); /* busy flag cannot be polled before this*/
46    LCD_command(0x38); /* set 8-bit data, 2-line, 5x7 font */
47    LCD_command(0x06); /* move cursor right after each char */
48    LCD_command(0x01); /* clear screen, move cursor to home */
49    LCD_command(0x0F); /* turn on display, cursor blinking */
50  }
51
52
53  void LCD_ready(void) {
54    char status;
55    /* change to read configuration to poll the status register */
56    GPIOC->MODER &= ~0x0000FFFF; /* clear pin mode */
57    GPIOB->BSRR = RS << 16; /* RS = 0 for status register */
58    GPIOB->BSRR = RW; /* R/W = 1 for read */
59
60    do { /* stay in the loop until it is not busy */
61    GPIOB->BSRR = EN; /* pulse E high */
62    delayMs(0);
63    status = GPIOC->IDR; /* read status register */
64    GPIOB->BSRR = EN << 16; /* clear E */
65    delayMs(0);
66    } while (status & 0x80); /* check busy bit */
67
68    /* return to default write configuration */
69    GPIOB->BSRR = RW << 16; /* R/W = 0, LCD input */
70    GPIOC->MODER |= 0x00005555; /* Port C as output */
71  }
72
73
74  void LCD_command(unsigned char command) {
75    LCD_ready(); /* wait for LCD controller ready */
76    GPIOB->BSRR = (RS | RW) << 16; /* RS = 0, R/W = 0 */
77    GPIOC->ODR = command; /* put command on data bus */
78    GPIOB->BSRR = EN; /* pulse E high */
79    delayMs(0);
```

```
  delayMs(0);
  GPIOB->BSRR = EN << 16; /* clear E */
}


/* This function is used at the beginning of the initialization *
when the busy bit of the status register is not readable. */
void LCD_command_noPoll(unsigned char command) {
  GPIOB->BSRR = (RS | RW) << 16; /* RS = 0, R/W = 0 */
  GPIOC->ODR = command; /* put command on data bus */
  GPIOB->BSRR = EN; /* pulse E high */
  delayMs(0);
  GPIOB->BSRR = EN << 16; /* clear E */
}


void LCD_data(char data) {
  LCD_ready(); /* wait for LCD controller ready */
  GPIOB->BSRR = RS; /* RS = 1 */
  GPIOB->BSRR = RW << 16; /* R/W = 0 */
  GPIOC->ODR = data; /* put data on data bus */
  GPIOB->BSRR = EN; /*pulse E high */
  delayMs(0);
  GPIOB->BSRR = EN << 16; /* clear E */
  printed_length++;
  if(printed_length == LCD_LENGTH) {
    LCD_command(0xC0);
  }
}


/* delay n milliseconds (16 MHz CPU clock) */
void delayMs(int n) {
  int i;
  for (; n > 0; n--)
  for (i = 0; i < 3195; i++) ;
}
```

```
/******************* GPIO Part *******************/

void PORTS_init(void) {
  RCC->AHBlENR |= 0x06; /* enable GPIOB/C clock */
  /* PB5 for LCD R/S */
  /* PB6 for LCD R/W */
  /* PB7 for LCD EN */
  GPIOB->MODER &= ~0x0000FC00; /* clear pin mode */
  GPIOB->MODER |= 0x00005400; /* set pin output mode */
  GPIOB->BSRR = 0x00C00000; /*turn off EN and R/W */

  /* ------------------------------------------------------------ */
  /* This function waits until LCD controller is ready to*/
  /* PC0-PC7 for LCD D0-D7, respectively. */
  GPIOC->MODER &= ~0x0000FFFF; /* clear pin mode */
  GPIOC->MODER |= 0x00005555; /* set pin output mode */
}

/* accept a new command/data before returns.
* It polls the busy bit of the status register of LCD controller. *
In order to read the status register, the data port of the *microcontroller has to change to an input port before reading * the
LCD. The data port of the microcontroller is return to * output port
before the end of this function.
*/
/* ------------------------------------------------------------ */
```

LCD.h

```
#ifndef INC_LCD_H_
#define INC_LCD_H_

#include "stm32f401xe.h"

#define RS 0x20 /* PB5 mask for reg select */
#define RW 0x40 /* PB6 mask for read/write */
#define EN 0x80 /* PB7 mask for enable */
#define LCD_LENGTH 16

void delayMs(int n);
void LCD_command(unsigned char command);
void LCD_command_noPoll(unsigned char command);
void LCD_data(char data);
void LCD_init(void);
void LCD_ready(void);
int digit_count(int num);
void LCD_print_number(int num);
void LCD_print_string(char str[]);
void LCD_clear(void);

void PORTS_init(void);

#endif /* INC_LCD_H_ */
```